

Positional Convolution Experts : Bilinear Routing with decorrelation Loss

Mirko Bonifazi

June 30, 2026

Contents

1	Architecture	2
1.1	Token Extraction	3
1.2	Convolution Experts and Dense-Block	3
1.3	Router Gate and Dispatch	5
1.3.1	Bi-Linear vs Additive Gate	5
2	Training	5
2.1	Decorrelation Loss	5

1 Architecture

Positional Convolution Experts (PCE) is an architecture inspired by ResNet. We replace each ResNet-Blocks with PCE-Blocks to transform ResNet into a MoE architecture. At the beginning of the architecture we insert a classical Stem-Block, which transform the input to first features map : we define $X^{(0)}$ as the first features map that is returned from Stem-Block $X^{(0)} = \text{SiLU}(\text{BN}(\text{Conv}_{7 \times 7, s=2}(X_{in})))$, where X_{in} is the input image, defined as tensor of size $3 \times H_{in} \times W_{in}$, while $X^{(0)}$ is a tensor of size $64 \times \frac{H_{in}}{2} \times \frac{W_{in}}{2}$

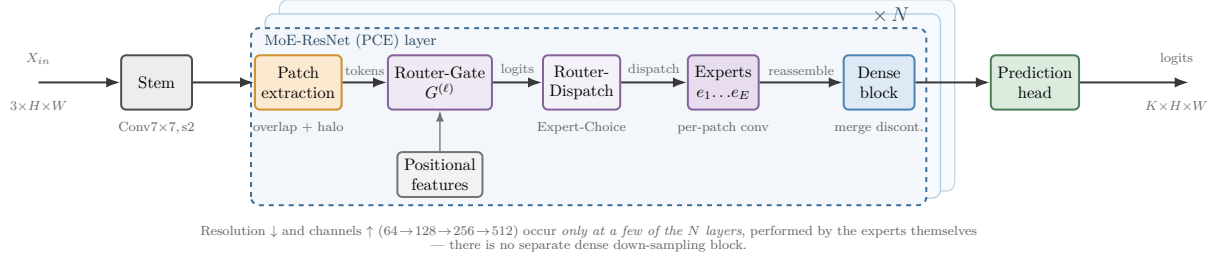


Figure 1: **PCE architecture overview.** A strided 7×7 stem maps the input image X_{in} to the first feature map. The network then repeats a single *MoE-ResNet* (PCE) layer N times: each feature map is cut into overlapping patches (with a halo for local context); the *Router-Gate* $G^{(\ell)}$ fuses the patch content (semantic branch) with content-free positional features into routing logits; the *Router-Dispatch* assigns tokens to experts by Expert-Choice; the experts $e_1, \dots, e_E$ transform their patches; and a dense block merges the inter-patch discontinuities. Down-sampling and channel doubling ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$) are *not* a separate dense module and do *not* happen at every layer: they are carried out by the experts at only a few of the N layers. After the final layer a convolutional prediction head produces the output.

Backbone is composed of different set of experts for each PCE-Blocks $\mathcal{E}^{(\ell)} = \{e_1^{(\ell)}, \dots, e_E^{(\ell)}\}$ and a dense-block. Each PCE-Blocks perform tokens extraction through a patch-extractor that cuts the previous feature map into patches of a specific size. To ensure a local context for each experts, that will process the tokens, we set a patch-overlap among adjacent patch. Following this, router-gate execute logits computation, Router-Gate is different for each PCE-Blocks. To compute the logits, it uses semantic and positional features, which interact through inner-product. The token dispatch is done by router $\mathcal{R}$ through Expert-Choice, each experts takes its token set $\mathcal{I}_{n,\epsilon}^{(\ell)}$ as input and process each tokens inside it. After that each experts has processed its tokens from token set $\mathcal{I}_{n,\epsilon}^{(\ell)}$, we aggregate experts output as a weighted sum, where the weights are the probability scores returned by the router, and reconstruct feature-map using the final experts outputs. After feature-map's reconstruction, model processes the reconstructed feature map with a dense-blocks to smooth the discontinuities introduced by the per-patch expert processing. Every two PCE-Blocks we do downsampling and double the number of channels, to achieve this we have implemented two different version : 1) First version uses a classical ResNet-Block to do downsampling and doubles of channels 2) Second version uses experts to do that.

After that each MoE-ResNet-Block is executed, there is a prediction head. We have tested PCE for two task : Image Segmantation and Image Classification, so the prediction head changes according to the task. For image classification we use a GAP layer and one linear-layer to project features map into the class space, if the task is segmentation, at the moment, we use a small convolution decoder with a parameter-free bilinear upsampling. In future we want to implement a full PCE encoder-decoder architecture, the encoder halves patch size and uses downsampling, while the decoder doubles patch size and uses upsampling, we connect encoder and decoder outputs like U-Net.

1.1 Token Extraction

Defines $X^{(\ell)} \in \mathbb{R}^{B \times C_\ell \times H_\ell \times W_\ell}$ as the feature map at the entrance to layer ℓ , where :

- B is the number of batch
- C_ℓ is the number of channel at layer ℓ
- H_ℓ, W_ℓ are the height and width of feature map at layer ℓ

For a single MoE-ResNet-Block we call its output as $X^{(\ell+1)} \in \mathbb{R}^{B \times C_\ell \times H_\ell \times W_\ell}$, if the blocks are normal, or else $C_{\ell+1} = 2C_\ell$, $H_{\ell+1} = \frac{H_\ell}{2}$ and $W_{\ell+1} = \frac{W_\ell}{2}$ if the blocks are a downsampling block. Patch Extractor, for each feature map $X^{(\ell)} \in \mathbb{R}^{B \times C_\ell \times H_\ell \times W_\ell}$ defines a grid of patches starting from the original feature map size. Number of patches long the height and width are $Q_{h,\ell} = \frac{H_\ell}{P_{h,\ell}}$, $Q_{w,\ell} = \frac{W_\ell}{P_{w,\ell}}$ correspondingly, where $P_{h,\ell}$ and $P_{w,\ell}$ are the size of patches for layer ℓ . For one image the total number of patches is $Q_\ell = Q_{h,\ell} \cdot Q_{w,\ell}$, consequently considering B as the batch size, the total number of token at the entrance of the layer ℓ is $N_\ell = B \cdot Q_\ell$. The patch dimension is halved proportionally to image resolution, as the depth increases H_ℓ and W_ℓ decrease together with $P_{h,\ell}$ and $P_{w,\ell}$ leaving invariant the rapport, it turns out that $Q_\ell = Q$, $N_\ell = N = BQ \forall \ell$. So the number of tokens is costant for all PCE-Blocks.

To ensure a local context at each experts we cut the patches with halo. Patches have size $P_{h,\ell} \times P_{w,\ell}$, to capture local context we extend the size of patches for a certain value that we call δ , so extended patch has size $\tilde{P}_{h,\ell} = P_{h,\ell} + 2\delta$ and $\tilde{P}_{w,\ell} = P_{w,\ell} + 2\delta$. Experts and router gate process the extended patch, so the real token that is processed by experts is $\tilde{t}_n^{(\ell)} \in \mathbb{R}^{C_\ell \times \tilde{P}_{h,\ell} \times \tilde{P}_{w,\ell}}$

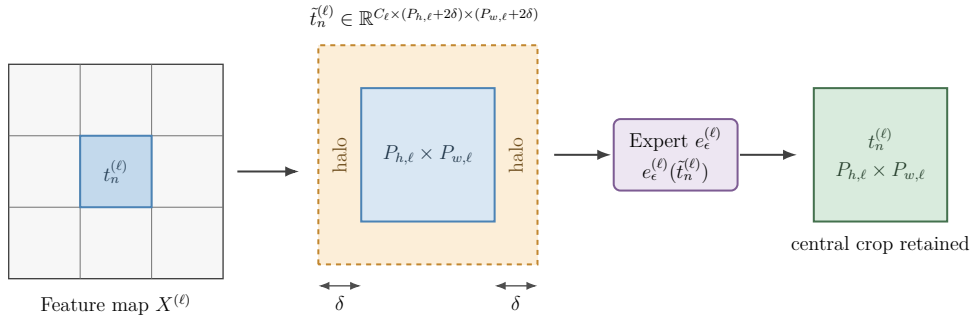


Figure 2: Overlapping patch extraction at layer ℓ . Each core patch (size $P_{h,\ell} \times P_{w,\ell}$) is extended with a halo of width δ on every side, producing the token $\tilde{t}_n^{(\ell)} \in \mathbb{R}^{C_\ell \times (P_{h,\ell} + 2\delta) \times (P_{w,\ell} + 2\delta)}$ actually processed by the expert $e_\epsilon^{(\ell)}$. Only the central region $t_n^{(\ell)} = \text{crop}_\delta(\tilde{t}_n^{(\ell)})$ of size $P_{h,\ell} \times P_{w,\ell}$ is retained during feature-map reconstruction.

After that experts have processed their tokens, we cut the patches' center, that we define as $t_n^{(\ell)} = \text{crop}_\delta(\tilde{t}_n^{(\ell)}) \in \mathbb{R}^{C_\ell \times P_{h,\ell} \times P_{w,\ell}}$, it corresponds with the original patch. To reconstruct the final feature map at the final of PCE-Blocks we use $t_n^{(\ell)}$. Once the Patch Extractor pulled out all patches from the feature map, it computes for each of them the positional features as $p_n^{(\ell)} \in \mathbb{R}^{C_p}$, obtained through a Fourier-Feature encoding. The global tensor of positional features is $P^{(\ell)} \in \mathbb{R}^{N \times C_p}$.

1.2 Convolution Experts and Dense-Block

The PCE-Blocks have its own experts set $\mathcal{E}^{(\ell)} = \{e_1^{(\ell)}, \dots, e_E^{(\ell)}\}$, each one of them have the same structure :

$$\text{Conv}_{3 \times 3} \rightarrow \text{GroupNorm} \rightarrow \text{SiLU} \rightarrow \text{Conv}_{3 \times 3} \rightarrow \text{GroupNorm}$$

with expert internal residual connection. Each experts has its independent parameters. Experts takes as input the extended token $\tilde{t}_n^{(\ell)}$, process it and return the processed extended token with residual connection $\tilde{y}_{n,e}^{(\ell)} = \tilde{t}_n^{(\ell)} + e_e^{(\ell)}(\tilde{t}_n^{(\ell)})$, then we crop experts output and obtain $y_{n,e}^{(\ell)} = \text{crop}_\delta(\tilde{y}_{n,e}^{(\ell)}) = t_n^{(\ell)} + \text{crop}_\delta(e_e^{(\ell)}(\tilde{t}_n^{(\ell)}))$. Now define the set of experts who have chosen token n as $\mathcal{I}_{n,\epsilon}^{(\ell)}$. The outputs' aggregation is computed as :

$$o_n^{(\ell)} = \begin{cases} \sum_{\epsilon \in \mathcal{I}_{n,\epsilon}^{(\ell)}} \pi_{n,\epsilon}^{(\ell)} y_{n,e}^{(\ell)} & \text{if } \mathcal{I}_{n,\epsilon}^{(\ell)} \neq \emptyset \\ t_n^{(\ell)} & \text{if } \mathcal{I}_{n,\epsilon}^{(\ell)} = \emptyset \end{cases}$$

where $\pi_{n,\epsilon}^{(\ell)}$ is the router dispatch score, i.e. how strongly expert ϵ prefers token n . If a token is selected by no expert, it is propagated through an identity connection, allowing gradient signals to flow to the preceding layers. Now that we have the aggregated output we have to reconstruct the feature map, placing each patch in its original coordinate : $F_{\text{sparse}}^{(\ell)} = \text{reassemble}(\{o_n^{(\ell)}\}_{n=1}^N)$ and then model processes it through the dense-block, which returns the next feature map $X^{(\ell+1)} = F_{\text{sparse}}^{(\ell)} + \text{DWConv}_{3 \times 3}^{(\ell)}(F_{\text{sparse}}^{(\ell)})$, this dense convolution allows local communication across patch borders. Dense convolution in dense block is deptwise.

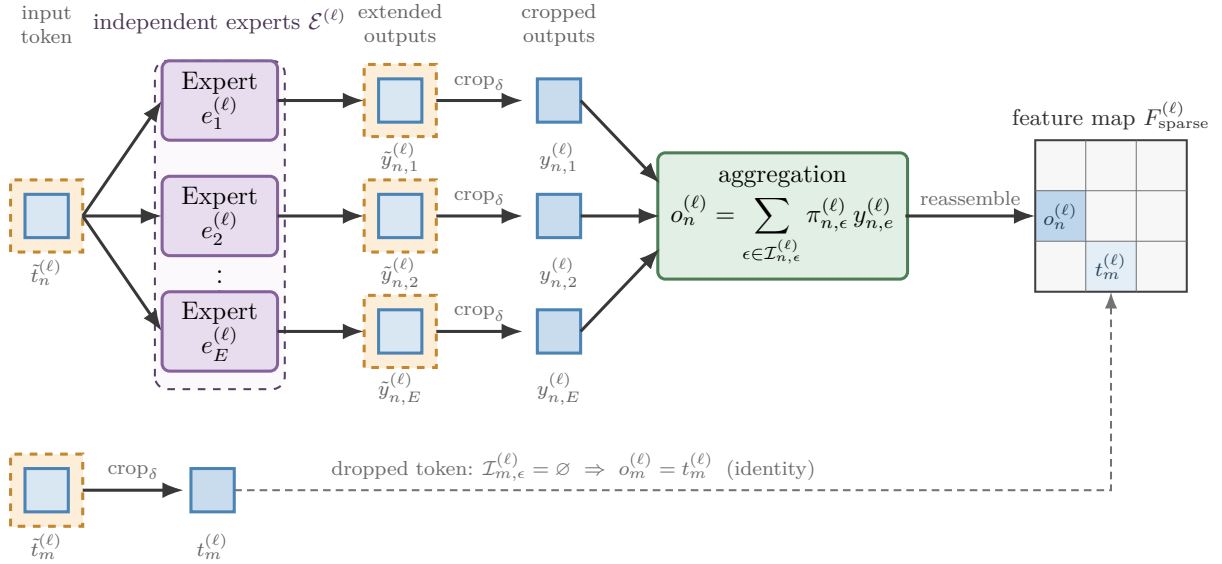


Figure 3: Convolution experts, aggregation and feature-map reconstruction at layer ℓ . The extended token $\tilde{t}_n^{(\ell)}$ is processed *independently* by the experts that selected it; each expert output $\tilde{y}_{n,e}^{(\ell)}$ is cropped to $y_{n,e}^{(\ell)}$ (the halo δ is removed). The cropped outputs are combined by the router-weighted aggregation $o_n^{(\ell)} = \sum_{\epsilon \in \mathcal{I}_{n,\epsilon}^{(\ell)}} \pi_{n,\epsilon}^{(\ell)} y_{n,e}^{(\ell)}$. A token selected by no expert ($\mathcal{I}_{m,\epsilon}^{(\ell)} = \emptyset$, e.g. m) is propagated through the identity, $o_m^{(\ell)} = t_m^{(\ell)}$. The reconstructed variables are reassembled at their original coordinates into the sparse feature map $F_{\text{sparse}}^{(\ell)}$.

After the model has executed all PCE-Blocks there is a prediction head, its structure depends on task, we have trained model for two task, Segmantation and Classification. Let $X^{(L)} \in \mathbb{R}^{B \times C_L \times H_L \times W_L}$ be the feature map produced by the last PCE-Block, C_L its number of channels, K the number of classes and $H_{in} \times W_{in}$ the input resolution. The segmentation head is :

$$\begin{aligned} X^{(L)} &\xrightarrow{\text{Conv}_{3 \times 3}} \text{BN} \rightarrow \text{SiLU} \rightarrow \text{Dropout}(0.1) \\ &\xrightarrow{\text{Conv}_{3 \times 3}} \text{BN} \rightarrow \text{SiLU} \rightarrow \text{Dropout}(0.2) \\ &\xrightarrow{\text{Conv}_{1 \times 1}} Z \in \mathbb{R}^{B \times K \times H_L \times W_L} \end{aligned}$$

with channel schedule $C_L \rightarrow C_L/2 \rightarrow C_L/4 \rightarrow K$, followed by a parameter-free bilinear upsampling to the input resolution $\hat{Y} = \text{Upsample}_{H_{in} \times W_{in}}^{\text{bilinear}}(Z) \in \mathbb{R}^{B \times K \times H_{in} \times W_{in}}$. When the task is classification, the prediction head is a global average pooling followed by a single linear layer :

$$z = \text{GAP}(X^{(L)}) \in \mathbb{R}^{B \times C_L}, \quad \hat{y} = Wz + b \in \mathbb{R}^{B \times K}, \quad W \in \mathbb{R}^{K \times C_L}.$$

1.3 Router Gate and Dispatch

Router-Gate is the hearth of our routing algorithm. It work in two different way : Semantic and Positional way. Router-Gate uses they interaction to compute router logits, then Router-Dispatch uses it to compute the probability scores that assigne tokens to experts who have chose them. Let $G^{(\ell)}$ be the Router-Gate at layer ℓ , every Router-Gate in all layers have the same structure and work. All $G^{(\ell)}$ have two different group of parameters, one group for semantic features $U^{(\ell)} \in \mathbb{R}^{E \times R \times C_\ell \times \tilde{P}_{h,\ell} \times \tilde{P}_{w,\ell}}$, and another one for positional features $V^{(\ell)} \in \mathbb{R}^{E \times R \times C_{p,\ell}}$ where R is the latent representation size. For one single expert the parameters are $U_{e,r}^{(\ell)}$ and $V_{e,r}^{(\ell)}$. Router-Gate takes $\tilde{t}_n^{(\ell)}$ as the semantic features and $p_n^{(\ell)}$ as the positional features, processing them Router-Gate returns two different logits :

$$z_{n,e,r}^{(\ell)} = \langle p_n^{(\ell)}, V_{e,r}^{(\ell)} \rangle, \quad a_{n,e,r}^{(\ell)} = \langle \tilde{t}_n^{(\ell)}, U_{e,r}^{(\ell)} \rangle$$

correspoondingly global tensors are $Z_{n,e}^{(\ell)} \in \mathbb{R}^R$ and $A_{n,e}^{(\ell)} \in \mathbb{R}^R$. To make interact them we build an expert-specific bi-linear matrix of max rank R :

$$W_e^{(\ell)} = U_e^{(\ell)\top} V_e^{(\ell)} = \sum_{r=1}^R u_{e,r}^{(\ell)} v_{e,r}^{(\ell)\top} \in \mathbb{R}^{C_{p,\ell} \times D_\ell}, \quad \text{rank}(W_e) \leq R$$

each elements' W_e represent how much, for the expert e , semantic and position must interact, we are measuring the expert activation based on how much that expert prefer certain semantic pattern component when it appears in a specific positional pattern. Each expert has its distinct matrix $W_1^{(\ell)}, \dots, W_E^{(\ell)}$, two expert can interpret differently the same pair $(\tilde{t}_n^{(\ell)}, p_n^{(\ell)})$, for example let's hypothesize that a certain semantic direction represent an ayes structure, one expert could learn a highly interaction when that semantic structure appears in the left face side. So the expert-specific bi-linear matrix improve the model capacity to learn different joint compatibility.

The logits are computed as:

$$L_{n,e} = \frac{1}{\sqrt{R}} (\tilde{t}_n^{(\ell)})^\top W_e^{(\ell)} p_n^{(\ell)} + b_e = (U_e \tilde{t}_n^{(\ell)})^\top (V_e p_n^{(\ell)}) = \langle A_{n,e}, Z_{n,e} \rangle$$

In the bi-linear matrix the rank specifies the number of channel in which the two input can to interact. After logits computation Router-Dispatch use the final logits $L_{n,e}$ to compute softmax distribution over token dimension $p = \text{softmax}(\frac{L_{n,e}}{\tau})$ where τ is a router temperature that decay during training, then each experts chose its prefers token, let $\pi_{n,e}$ be the probability score with which expert e would takes token n . Each experts has its token capacity, determines how many tokens can they takes : $K = \min(N \lceil c \frac{E}{N} \rceil)$, where c is the capacity-factor.

1.3.1 Bi-Linear vs Additive Gate

2 Training

2.1 Decorrelation Loss